

Set

1. Introduction

A **set** in Python is an **unordered, mutable collection of unique elements**. Sets automatically remove duplicates and are mainly used for **membership testing**, **removing duplicates**, and performing **mathematical set operations** like union, intersection, etc.

2. Definition of a Set

A set is defined using **curly braces { }** or the **set() constructor**.

Example:

```
s1 = {1, 2, 3, 4}
s2 = {"apple", "banana", "mango"}
s3 = {10, "hello", 3.14, True}
```

Single Element Tuple:

```
s = {} -> If you define like this then it is dictionary not a set
s = set() -> You have to write like this for an empty set
```

3. Characteristics of Set

- **Unordered:** Does not preserve insertion order
- **Mutable:** You can add or remove elements
- **Heterogeneous:** Accepts mixed data types
- **Does not allow duplicates:** Automatically removes repeated items
- **No indexing:** You cannot access items using index like lists/tuples
- **Faster for membership testing:** Very efficient for checking if an item exists

4. Accessing Set Elements

Sets **do not support indexing or slicing** because they are unordered.

You can only access elements using:

- **Loop**
- **Membership tests**

```
s = {10, 20, 30}

for x in s:

    print(x)

print(20 in s)  # True
```

5. Set comprehension:

Set comprehension is a short and clean way to create a set using a loop in **one line**.

It uses curly braces { } and automatically removes duplicates.

```
Ex: s = {x for x in range(5)}

print(s)  # Output: {1, 2, 3, 4, 5}
```

6. Set Methods

A) Adding Methods:

1) **add()** : Adds a single element

```
Ex: s = {1, 2}

s.add(3)
```

2) **update()** : Adds multiple elements

```
Ex: s.update([4, 5, 6])
```

update() takes an iterable (list, set, tuple, string, etc.) and adds each element individually to the set.

B) Removing Methods:

1) **remove()** : Adds a single element

```
Ex: s.remove(2)
```

2) discard() : Removes element; **no error** if not found

Ex: `s.discard(10)`

3) pop() : Removes a random element

Ex: `s.pop()`

4) clear() : Removes all elements

Ex: `s.clear()`

C) Another important set functions:

1) len() : Gives length of the set

Ex: `s = {10, 20, 30}`
`print(len(s))` # Output: 3

2) max(): Gives maximum value from set

Ex: `s = {10, 20, 30}`
`print(max(s))` # Output: 30

3) min() : Returns the minimum element.

Ex: `s = {10, 20, 30}`
`print(min(s))` # Output: 10

4) sum() : Removes all elements

Ex: `s = {10, 20, 30}`
`print(sum(s))` # Output: 60

5) sorted() : Returns a **list** of sorted elements (set remains unchanged).

Ex: `s = {5, 1, 7, 3}`
`print(sorted(s))` # Output: [1, 3, 5, 7]
`print(s)` # Original set => {1,3,5,7} (order not guaranteed)

6) any(): Returns **True** if at least one element is True (non-zero).

Ex: s = {0, "", False, 5}

print(any(s)) # Output: True (because 5 is True)

7) all() : Returns **True** only if all elements are True (non-zero).

Ex: s = {1, 2, 3}

print(all(s)) # Output: True

s2 = {0, 2, 3}

print(all(s2)) # Output: False (because 0 is False)

8) set() : Converts list/tuple/string to set.

Ex: l = [1, 2, 2, 3]

print(set(l)) # Output: {1,2,3}

st = "hello"

print(set(st)) # Output: {'h','e','l','o'}

9) enumerate() : Used to loop with index (though set is unordered).

Ex: s = {"apple", "banana", "mango"}

for index, value in enumerate(s, start=1):

print(index, value)

#Output : 1 mango

2 apple

3 banana

10) type() : Used to check data type.

Ex: s = {1, 2, 3}

print(type(s)) # Output: <class 'set'>

11) copy() : Returns a shallow copy of set.

```
Ex: a = {1,2,3}

b = a.copy()

print(b)    # {1,2,3}
```

D) Mathematical Set Operations Function:

1) union() : It combines all unique elements from both sets

```
Ex: a = {1, 2}
b = {2, 3}
print(a | b) (symbolic union) # {1,2,3}
print(a.union(b))  # {1,2,3}
```

2) intersection() : It gives the common elements from both sets

```
Ex: a = {1, 2, 3}
b = {2, 3, 4}
print(a & b)  #{2,3}
print(a.intersection(b))  # {2,3}
```

3) difference(): It subtracts one set from another (elements in A but not in B)

```
Ex: a = {1, 2, 3}
b = {2, 3}
print(a - b)  #{1}
print(a.difference(b))  # {1}
```

4) symmetric_difference() : It gives elements not common in both sets

```
Ex: a = {1, 2, 3}
b = {3, 4}
print(a^b)  #{1, 2, 4}
print(a.symmetric_difference(b))  # {1,2,4}
```

5) update() : Same as **union**, but it **changes the original set**.

```
Ex: a = {1, 2}
    b = {2, 3, 4}
    a.update(b)
    print(a)    # {1, 2, 3, 4}
```

6) intersection_update() : This is like **intersection**, but it **updates the original set**.

```
Ex: a = {1, 2, 3}
    b = {2, 3, 4}
    a.intersection_update(b)
    print(a)    # {2, 3}
```

7) difference_update(): This is like **difference**, but it **changes the original set**.

```
Ex: a = {1, 2, 3, 4}
    b = {3, 4}
    a.difference_update(b)
    print(a)    # {1, 2}
```

8) symmetric_difference_update() : This is like **symmetric_difference**, but it **changes the original set**.

```
Ex: a = {1, 2, 3}
    b = {3, 4}
    a.symmetric_difference_update(b)
    print(a)    # {1, 2, 4}
```

E) Relation tests:

1) `issubset()` : Checks if all elements of A are in B.

Ex: `a = {1,2}`

`b = {1,2,3}`

`print(a.issubset(b))` # True

2) `issuperset()` : Checks if A contains all elements of B.

Ex: `a = {1,2,3}`

`b = {1,2}`

`print(a.issuperset(b))` # True

3) `isdisjoint()` : True if sets have **no common elements**.

Ex: `a = {1,2}`

`b = {3,4}`

`print(a.isdisjoint(b))` # True